

## E09 · $\Theta$ -경로 — 가중치에새기기

이 권은 본문 ch19(LoRA 와 모델 편집), ch20(SEAL), ch21(Language Models Need Sleep)을 수식과 증명 없이 다시 설명한다. 세 장이 다루는 것은 세경로 중 마지막 하나다 — 모델 밖도 아니고 요청 중에만 사는 상태도아니라, 배포되는 가중치 자체에 적어 넣는 방식이다.

이 경로에는 앞의 두 경로에 없는 성질이 하나 있고, 이 권은 그 성질을 향해간다. 여기서는 쓰기가 끝나면 `.safetensors` 파일이 달라진다. 그 한 문장에서 개인화 가능성, 배칭 가능성, 롤백 단위, 캐시 수명이 전부 같린다. 그리고 이 경로의 논문들이 공통으로 재지 않은 값이 하나 있다 — 반복해서 써도 되는가. 이 권은 그 두 가지를축으로 읽는다.

### 1. 마지막 층 — 고치면 배포가바뀐다

이 라인의 모든 논문은 “질의가 오기 전에 계산을 미리 해서 상태를 바꿔둔다”는 한 문장으로 환원된다. 바꿀 수 있는 상태가 셋이고, 이 권은 그중셋째를 다룬다.

**기호 풀이**  $\Theta$ (slow weights) = 배포되는 가중치. 우리가 굵고 HBM에 올리는텐서 전부다.  $W$ (fast weights) = 요청을 처리하는 동안만 살아 있는 고정크기 상태.  $E$ (external store) = 모델 밖의텍스트 · 벡터 · 그래프 저장소.

$E$ 를 고치면 서빙되는 모델은 그대로다.  $W$ 를 고치면 요청이 끝날 때같이 사라진다.  $\Theta$ 를 고치면 배포 아티팩트가 바뀐다. 이차이가 크기의 차이가 아니라 종류의 차이라는 것이 이 권의 출발점이며, 그래서 이 경로를  $\Theta$ -경로라고 따로 부른다.

쓰기는 한 줄이다.

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s) \quad (\text{U-}\Theta)$$

**기호 풀이**  $k$  = sleep 라운드 첨자. 유희 시간에 도는 정리 작업의회차이며, 토큰 첨자  $t$ 와는 완전히 다른 시계다.  $\eta_{\Theta}$  = sleep 학습률. 요청 안에서 도는 학습률  $\eta_t$ 와 반드시구분한다.  $\mathcal{L}$  = 학습에 쓰는 loss.  $\mathcal{R}_k$  = 이라운드가 학습에 쓸 데이터 집합.  $\mathcal{H}_k$  = 그때까지 쌓인 경험.  $\text{gen}(\cdot; B_s)$  = 예산  $B_s$ 를 써서 그 경험으로부터 학습집합을 만드는 절차.

이 식이 말하는 것은 두 단계라는 것이다. 먼저 무엇으로학습할지를 만들고 (gen), 그 다음 그것으로 가중치를 민다. 이권의 세 장이 갈리는 지점은 전부 이 둘 중 하나다 — ch19는 미는 방법만다루고 만드는 단계가 비어 있으며, ch20과 ch21은 그 빈칸을 채운다.

비용은 언제나 네 값으로 켜다.

**기호 풀이**  $B_s$  = sleep 계산 예산 (FLOPs 또는 tokens).  $L_w$  = wake 지연기여 (ms, TTFT/ITL) — 사용자가 실제로 기다리는 시간.  $C_{\text{cap}}$  = 상태 용량 (bytes).  $\rho$  = 망각 · 열화율, 즉 라운드를 반복했을 때 라운드당 잃는 성능.

이 네 칸이 이 권의 채점표다. 미리 말해 두면, 이 권의 논문들에서 가장자주 비어 있는 칸은  $\rho$ 이고 그것이 하필 이 경로에서 가장 중요한칸이다.

자세한 것은 ch11 § 11.2, ch19 § 19.1.

## 2. 두 가지 쓰기 — 남기느냐 흡수시키느냐

$\Theta$ 에 쓰는 방법은 두 계열이다. 그리고 둘을 가르는 것은 정확도도 rank도 아니라 **쓰고 나서 무엇이 남는가**다.

**delta 방식**은 갱신을 base 가중치와 따로 저장할 수 있는 객체로 만든다. LoRA가 그 대표다. 아이디어는 한 줄로 끝난다 — 변화량이 아무모양이나 될 수 있게 두지 말고, **얇은 두 판의 곱**으로만 표현되게 강제한다.

**기호 풀이**  $r = \text{LoRA rank}$ . 두 판 사이의 두께이며 이 책 전체에서  $r$ 은 언제나 rank다 (보상은  $R$ 로 쓴다).  $A$ 는  $r \times d_{\text{in}}$ ,  $B$ 는  $d_{\text{out}} \times r$  모양이고, 실제로 base에 더해지는 것은 그 곱  $BA$ 다.  $S = \text{LoRA}$ 를 붙이는 행렬들의 집합 — 예컨대 attention의 query와 value projection 둘.

두께  $r$ 이 작으면 저장할 것도 작아진다. GPT-3 175B에  $r = 4$ 로 query·value 두 자리에만 붙이면 체크포인트 **350GB가 delta 35MB**가 되고, optimizer가 들고 있어야 하는 VRAM도 1.2TB에서 350GB로 준다.

**직접 편집 방식**은 지정한 가중치 행렬을 제자리에서 고쳐 쓴다. ROME과 MEMIT이 그렇다. 갱신이 base에 흡수되므로 따로 남는 파일이 없고, 편집을 100개 하든 10,000개 하든 체크포인트 크기가 같다.

여기서 흔한 오해 하나를 끊어 둔다. **갈림길은 저랭크냐 아니냐가 아니다**. MEMIT의 갱신도 구조상 저랭크이고, LoRA도 merge를 실행하면 결국 base에 흡수되어 같은 자리에 도달한다. 갈림길은  $(A, B)$ 를 그대로 보관하느냐다. 보관하면 언제든 뺄셈 한 번으로 되돌릴 수 있고, 흡수시키면 되돌리는 연산이 존재하지 않는다.

**비유** delta 방식은 책에 포스트잇을 붙이는 것이고, 직접 편집은 활자를 다시 짜서 그 판으로 다시 찍는 것이다. 포스트잇은 떼면 되고 사람마다 다른 것을 붙일 수 있다. 다시 짰판은 그 판으로 찍은 모든 책에 똑같이 적용된다. — **비유가 깨지는 곳**: 포스트잇은 읽을 때 눈이 한 번 더 가지만 merge된 delta는 읽는 비용이 정확히 0이다. 연산 그래프가 base와 글자 그대로 같아지기 때문이다. 그래서 이 대비를 “편한 대신 느리다”로 옮기면 틀린다. 두 방식은 속도에서 갈리지 않고 **되돌리기와 소유권**에서 갈린다.

**시스템 모델링 관점** 이 갈림길이 서빙에서 무엇을 바꾸는지가 이 권에서 가장 실무적인 대목이다. 세 항으로 갈린다.

**첫째, wake 비용은 두 방식 모두 0이다**. merge된 LoRA는 GEMM 모양이 base와 동일하고, 직접 편집도  $\Theta \leftarrow \Theta + \Delta$ 라는 같은 모양의 제자리 덧셈이므로 파라미터 수·연산 그래프·토큰당 메모리 트래픽이 편집 전과 같다. 다만 두 편집 논문은 이 사실을 **주장조차 하지 않으며** 추론 지연을 한 번도 재지 않았다. 구조가 뻗은 것과 논문이 켜진 것은 다르다.

**둘째, 재배포되는 단위가 다르다**. delta 방식의 단위는 base 350GB와 별개인 35MB 파일이다. base는 HBM에 상주한 채로 두고 갈아끼우는 것은 35MB 뿐이다. 직접 편집의 단위는 **체크포인트 전체**다. 그래서 “checkpoint 크기 =  $C_{\text{cap}}$ ”이라는 대응이 이 방식에서는 무의미해진다 — 상태량이 0으로 보이는 대신, 편집 집합이 다른 두 배포본이

완전히 별개의 체크포인트가 된다.

셋째, 그리고 이것이 결정적인데 — 개인화하려면 **delta** 여야 한다. 직접 편집은 공유 가중치를 제자리에서 고치므로 애초에 분리 가능한 아티팩트가 생기지 않는다. 사용자마다 다른 사실을 넣으려면 사용자마다 모델 **전체 사본**이 필요하다. 반대로 편집이 전역이면 — 10,000 개 사실이 모든 사용자에게 같은 사실이면 — 가중치 한 벌을 계속 공유하므로 shared-weight batching 이 온전히 살아남는다. 즉 두 방식은 같은  $L_w = 0$  을 정반대 대가로 산다. delta 는 아티팩트를 남겨되돌리기 · 개인화 · 다중 배포를 얻는 대신 요청마다 다른 가중치를 감수하고, 직접 편집은 아티팩트를 없애 배칭을 지키는 대신 되돌리기와 개인화를 통째로 포기한다. **이 갈림길은 알고리즘의 세부가 아니라 서빙 구조의 분기점이며, 두 편집 논문은 이 구분을 제기조차하지 않는다.**

자세한 것은 ch03 § 03.5, ch19 § 19.2, § 19.7.

### 3. 학습률이 없는 쓰기

ROME 이 이 권에서 개념적으로 가장 낮은 물건이다. 이 갱신에는  $\eta_\theta$  가 없다. gradient step 도 없고 반복도 없다. 한 번의 대수계산으로 답이 나온다.

방법은 이렇다. 특정 MLP 층의 행렬 하나를 “키를 넣으면 값이 나오는 표” 로 본다. 이미 들어 있는 키-값 대응은 되도록 흐트러뜨리지 않으면서, 새 키 하나에 대해서만 원하는 값이 정확히 나오도록 강제한다. 이 조건을 만족하는 최소 변화량은 닫힌 식으로 구해지고, 그 변화량은 **rank 1** 이다 — 벡터 두 개의 바깥곱 하나가 전부다.

두 가지가 계산에 들어간다. 하나는 기존 키들의 2 차 통계이고, Wikipedia 2020-05-01 스냅샷에서 뽑은 hidden state 100,000 개로 **모델당 한번** 추정해 캐시한 뒤 이후 모든 편집이 같은 것을 재사용한다. 다른 하나는 새 키에 대응시킬 목표 값 벡터인데, 이것만 최적화로 구한다. 여기서 최적화 변수는 **가중치가 아니라 활성 벡터 하나**다. 그 최적화에도 학습률이 있지만 (0.5), 그것은 활성 벡터를 미는 학습률이 아니라  $\Theta$  를 미는 학습률이 아니다. 둘을 같은 것으로 옮겨 적으면 틀린다.

**비유** 스프레드시트에서 셀 하나의 값을 고치되, 그 셀을 참조하는 다른 수식들이 최대한 덜 흔들리도록 고치는 것과 같다. 어디를 고칠지 정해지면 얼마나 고칠지는 계산으로 나오지 시행착오로 나오지 않는다. — **비유가 깨지는 곳**: 스프레드시트는 어느 셀이 어느 수식에 걸리는지 정확히 알지만, 여기서는 그 의존 관계를 아무도 모른다. “다른 기억과의 간섭을 최소화한다” 는 말이 뜻하는 것은 **추정된 키 분포에 대해 평균적으로** 최소화한다는 것뿐이고, 개별 사실이 실제로 무사한지는 편집한 뒤에 재 봐야 한다.

MEMIT 은 같은 구조에서 두 가지를 바꾼다. 첫째, 새 사실 하나만 강제하던 등식 조건을 목적함수 안으로 넣어 **수천 개를 한꺼번에** 밀어 넣을 수 있게 한다. 이때 옛 지식은 명시적인 replay 데이터 없이 키 분포의 2 차 통계만으로 보호되며, 얼마나 세게 보호할지를 정하는 손잡이  $\lambda$  가 붙는다 (전형값  $1.5 \times 10^4$ ). 이  $\lambda$  가  $\Theta$  -경로에서 유일하게 명시적인 **retention** 손잡이다 — 크게 잡으면 갱신이 0 으로 수렴하고, 작게 잡으면 새것이 잘 들어가는 대신 옛것이 밀린다.

둘째, 쓰기를 층 하나가 아니라 임계 MLP 층 여러 개에 나눠 싣는다. 각 MLP 가 최종 은닉 상태에 덧셈으로 기여하므로 변화량을 쪼개 실을 수 있고, 분배는 남은 층 수로 균등하게

나눈다. 그런데 순서가 본질이다. 한층을 고치면 그 위 층이 보는 활성이 전부 바뀌므로, 층을 오름차순으로돌면서 층마다 활성을 다시 수집해야 한다. 이 직렬 구간이 배치화가불가능한 유일한 축이다.

자세한 것은 ch19 § 19.3.1, § 19.3.2.

---

## 4. 쓸 자리는 사람이정한다

두 편집 논문 모두 “어디에 쓸 것인가” 를 학습하지 않는다. 계측기로 재고사람이 읽어 고정한다.

계측 방법은 세 번 돌리는 것이다. 정상으로 한 번, 주어 토큰을 망가뜨려 한번, 망가뜨린 상태에서 특정 은닉 상태만 정상 값으로 되돌려 한 번. 세번째에서 정답이 얼마나 복구되는지가 그 자리의 기여도다. GPT-2 XL 에서최대 기여는 **15 번째 층·주어의 마지막 토큰**에 있고 (8.7%), 같은 자리에서 MLP 몫이 6.6%, attention 몫이 1.6%다. 이지도를 보고 편집 층을 18 로 고정한다. MEMIT 은 같은 계측을 다시 수행하되층 하나가 아니라 범위를 고른다 — GPT-J 는 3-8 층, GPT-NeoX 는 6-10 층이다.

**모델이 바뀌면 사람이 다시 정해야 한다.** 자동화된  $\Theta$  쓰기파이프라인을 상상한다면 이 단계가 사람 손에 남아 있다는 사실이 그대로계약이 된다.

주소 규칙이 보편적이지도 않다. 대개 주어의 마지막 토큰이 결정적이지만, Windows Media Player에서는 ‘Windows’ 가, Madame de Montesson에서는 ‘Madame’ 이 결정적이다. 주어 안에 결정적 토큰이 아예 없는 사례도 있고, 마지막 주어 토큰의 효과가 음수인 사례도 있다. **논문은 그비율을 보고하지 않는다.** 자동 파이프라인의 언어로 옮기면 이것은 주소실패율이며, 값이 알려져 있지 않다.

한 가지 더. 이 편집은 **방향을 갖는다**. “시애틀의 랜드마크는스페이스 니들이다” 와 “스페이스 니들은 시애틀의 랜드마크다” 는 따로저장되므로 둘 다 바꾸려면 두 번 편집해야 한다. 상태량이 사실 개수가아니라 **연관 방향 개수**로 스케일한다는 뜻이다.

자세한 것은 ch19 § 19.3.3, § 19.8.1.

---

## 5. 얼마나 넣을 수 있나 — 10,000 은 상한이 아니라하한이다

MEMIT 의 기여를 한 줄로 줄이면 **한 번의 쓰기에 묶어 넣을 수 있는사실의 개수**를 1 에서 10,000 으로 키운 것이다.

**정의. 편집 용량의 실측 상한**은, 한 번의 쓰기에 써넣었을때 세 지표 — 새 사실이 들어갔는가, 바뀌 물어도 나오는가, 건드리지 않은것이 무사한가 — 가 동시에 무너지지 않는 최대 편집 수다. 이 값은아키텍처의 성질이 아니라 **쓰기 알고리즘의 성질**이다.

중요한 것은 10,000 이 붕괴점이 **아니라**는 사실이다. 실험이 로그눈금으로 1 부터 10,000 까지 훑고 거기서 끝나는데, 끝점에서 아직 무너지지않았다. 비교 기법의 붕괴점은 명시되어 있다 — ROME 은 10 개까지 잘 작동하고 32 개부터 열화하며, MEND 는 6 개에서 급락해 1,000 개 전에 새 사실을 넣는능력을 전부 잃는다. 즉 **10,000 은 관측된 하한**이다. 진짜 천장은아무도 재지 않았다.

대가는 표에 그대로 있다.

표 E09-1. GPT-J 6B에 10,000 개 사실을 한 번에 편집했을 때 [MEMIT Table 2, CounterFact]

| 항목                      | 편집 없음       | MEMIT       |
|-------------------------|-------------|-------------|
| 종합점수                    | 22.4        | 85.8        |
| 새 사실이 들어갔나              | 15.2        | 98.9        |
| 바뀌 물어도 나오나              | 17.7        | 88.6        |
| <b>건드리지 않은 이웃이 무사한가</b> | <b>83.5</b> | <b>73.7</b> |
| 문장이 무너지지 않았나            | 622.4       | 619.9       |

**편집하지 않은 이웃 주어의 정답률이 10pt 가까이 떨어진다.** GPT-NeoX 20B 에서도 같은 방향이다 (81.6  $\rightarrow$  70.8). 이것이  $\Theta$ -경로 대량 쓰기의 실측 부작용이며, 이 권에서 유일하게 수치로 잡힌 부작용이기도 하다.

다른 벤치마크에서는 더 나쁘다. zsRE 에서 MEMIT 의 종합점수는 50.7 인데, 분해하면 새 사실 삽입 96.7·바뀌 묻기 89.7·**이웃 보존 26.6** 이고 조화평균이 그 낮은 항에 지배된다. 그런데 그 26.6 은 **편집하지 않은 GPT-J 의 27.0 보다 낮다.** 논문은 이 격차를 설명하지 않고 “bleedover 가 최소” 라고만 쓴다.

기준선과의 비교에도 유보가 붙는다. 순진한 fine-tuning 이 새 사실을 넣는 능력에서는 MEMIT 을 앞서고 (99.4 대 98.9) 시간도 약 15.6 배 빠르다. MEMIT 의 우위는 삽입이 아니라 이웃 보존 (73.7 대 46.9) 과 문장 유창성 (619.9 대 293.9) 에 있다. 그리고  $\lambda$  를 키우면 앞의 둘이 좋아지는 대신 삽입능력이 떨어진다 — **모든 축에서 좋은  $\lambda$  는 없다.**

평가 범위도 좁다. **일반 능력 평가가 전혀 없고**, 저자가 지식범위를 명시한다 — 다루는 것은 방향성 있는 (주어, 관계, 목적어) 삼중항뿐이며 공간·시간 추론도, 수학도, **심지어 대칭관계도** 다루지 못한다. 즉 “10,000 개 memory” 는 10,000 개 삼중항이지 일반적인 의미의 기억 10,000 개가 아니다.

자세한 것은 ch19 § 19.6.1, § 19.6.2.

## 6. 쓰기는쌌다. 검증은 세지않았다

$\Theta$ -경로의 쓰기 비용이 얼마인지 이 권에서 유일하게 숫자로 답하는 논문이 MEMIT 이다. GPT-J 6B 에 10,000 개 편집이 **총 7.44 시간**이고, 그 안에서 목표 활성 벡터를 구하는 단계가 23,546.65 초, 나머지 가중치 갱신이 3,226.35 초다. 나누면 편집당 2.6773 초이고 첫 단계가 총 시간의 87.95%를 차지한다. 대조군인 ROME 은 같은 일에 44,248.26 초, 편집당 약 4 초다.

여기서 상각의 모양이 앞의 두 경로와 완전히 달라진다.  $E$ -경로에서는 “한 문맥을 몇 개의 질의가 공유하는가” 가 분모였다.  $\Theta$ -경로에서 편집은 wake 비용을 전혀 바꾸지 않으므로, 한 번 지불하고 이후의 **모든** 질의에 상각된다. 분모가 무한으로 가고 손익분기가 정의되지 않는다.

**비유** 정기권이 아니라 개축이다. 정기권은 몇 번 타야 본전인지 셀 수 있지만, 벽을 한번 옮기면 그 다음부터는 그냥 그 집이다. — **비유가 깨지는 곳**: 개축한 집은 도면과 시공 기록이 남아 원상복구가 가능하지만, 직접 편집에는 취소 연산이 없다. 롤백은 이전 체

크포인트로 통째로 교체하는것뿐이다.

그래서  $\Theta$ -경로에서 상각의 실제 분모는 질의 수가 아니라 **한 번의 sleep 라운드에 묶여 넣은 편집 수**다. 이 언어로 옮기면 MEMIT의 기여는 그분모를 1에서 10,000으로 키운 것이 된다.

그런데 회계에서 통째로 빠진 항이 있다. **쓰기 검증이다**. ROME은 사람 15명이 반사실 시나리오 50개를 평가했고, 편집 후 모델이 근거 없는 새사실을 지어낸다고 인정한다. 사람 평가가 자동 지표를 뒤집기도 한다 —평가자는 ROME을 경쟁 기법보다 삽입된 사실에 일관적이라고 볼 확률이 1.8배 높지만 **더 유창하다고 볼 확률은 1.3배 낮고**, 저자는 이 손실이 “우리의 다른 지표에 포착되지 않는다”고 쓴다. 그 검증 비용은 어떤 비용수치에도 들어 있지 않다.

“2초”는 쓰기 연산의 비용이지 올바르게 썼는지 확인하는 비용이 아니다.  $\Theta$ -경로에서 쓰기가 싸다는 인상은 검증을 회계 밖에 둔 결과다.

**시스템 모델링 관점** 편집이 서빙 함대에 실제로 얹힌다고 하면 세 가지가 따라 나온다. 셋 다 논문에 없다.

**첫째, prefix cache 전면 무효화 이벤트다**. MEMIT은 GPT-J의 3-8층 MLP를 고치므로 그 층 이후로 캐시되어 있던 KV가 전부 stale해진다. prefix cache를 대량으로 운용하는 스택에서  $\Theta$  갱신은 캐시를 통째로 버리는 사건이며, 이것이  $\Theta$ -경로 배포의 숨은 운영 비용이다. 두 편집 논문 모두 KV cache라는 단어를 한 번도 쓰지 않는다.

**둘째, sleep 라운드는 서빙과 같은 커널로 돌지 않는다**. MEMIT은 GPT-NeoX 편집에 “float16으로 모델을 돌릴 48GB GPU 하나와, 편집 방법을 실행할 조금 작은 GPU 하나”가 필요하다고 쓴다. 게다가 키 통계는 fp32로 수집·저장하는데 모델은 float16으로 돈다. 편집기의 **peak memory 발자국이 모델과 같은 급이다**. 유틸 서빙 GPU에 sleep을 슬쩍 얹는 그림은 이 수치와 맞지 않는다.

**셋째, 갱신 주기는 일 단위다**. 10,000 사실에 7.44시간이고, 논문의 뉴스 신선도 데모도 며칠 단위 서술이다. 하루 한 번 배치가 현실적 상한이며 실시간 갱신은 이 방법의 범위 밖이다.

$E$ -경로와 나란히 놓으면 대비가 선명하다. 편집 이후의 질의는 검색대역폭을 0지불하고,  $E$ -경로는 질의마다 지불한다. 대신 지식하나를 바꾸는 값은 편집이 2.6773초,  $E$ -경로가 거의 0이다. 쓰기가 비싸고 읽기가 공짜인 층 대 쓰기가 싸고 읽기가 유료인 층 — 이것이 세 경로 비용 분해의 뼈대다.

자세한 것은 ch19 § 19.5, § 19.7.

## 7. 여기까지의 공백 — 무엇을 쓸지는 아무도 주지않았다

ch19의 세 도구를 식 ( $U-\Theta$ )에 대보면 한 자리가 비어 있다.  $\text{gen}(\cdot; B_s)$ 가 **셋 다 항등이다**. LoRA가 받는 학습집합은 GLUE·WikiSQL 같은 기성 dataset이고, ROME과 MEMIT이 받는 것은 zsRE·CounterFact의 레코드다. 무엇을 쓸지는 언제나밖에서 주어진다.

이것이 왜 문제인가. 이 책이 sleep-time compute이라고 부르려면 네 조건을 통과해야 하고, 그중 두 번째가 **예산  $B_s$ 를 실제로 태워무언가를 만드는 단계가 있는가**이다. 도구만 있고 만드는 단계가 없으면 그조건이 성립하지 않는다. 그래서 ch19의 세 논문은  $\Theta$ -경로의 primitive를 세우지만 아직 sleep-time compute이 아니다.

덧붙일 것이 하나 더 있다. 세 논문 모두 **sleep 라운드**가  $k = 1$  에서 멈춘다. ROME 은 정의상 한 번에 하나만 편집하고, MEMIT 은 배치편집을 한 번 실행하고 끝나며, LoRA 는 delta 하나를 학습하고 두 delta 를누적하는 실험을 하지 않는다. **그러므로  $\rho$  를 켤 실험이 이 셋어디에도 없다.** 이 권의 나머지가 그 빈칸 둘 — 무엇을 쓸 것인가, 그리고반복해도 되는가 — 을 차례로 다룬다.

자세한 것은 ch19 § 19.4, ch11 § 11.3.

## 8. SEAL — 모델이 자기 학습 데이터를 쓴다

SEAL의 아이디어는 그 빈자리에 모델 자신을 놓는 것이다. 새 문맥이도착하면 모델이 먼저 **그 문맥으로 자기를 어떻게 학습시킬지를 글로쓴다**. 그 글이 학습 집합이 되고, 그것으로 가중치를 민 결과의 점수가 다시그 글을 쓴 정책의 보상이 된다.

**정의. self-edit** 은 모델이 자기 가중치 갱신의 내용을지정해 쓴 자연어 산출물이다. 최소 형식은 갱신에 쓸 학습 집합 그자체이고, 최대 형식은 거기에 optimizer 설정 (학습률 · epoch 수 · loss 를 어디에 걸지) 과 데이터 변환 도구 호출까지 포함한다. 식  $(U-\Theta)$  의 언어로 옮기면,  $\text{gen}(\cdot; B_s)$  이 **decode pass** 한 번이 된 것이다.

실증된 형식은 둘이다.

**형식 A — 데이터.** 지식 통합 실험에서 self-edit 은 passage 의 “함의” 목록이다. 프롬프트는 “다음 글을 읽고 내용에서 직접 또는 간접으로따라 나오는 함의 목록을 만들어라” 한 줄이며, 생성된 문장들이 줄 단위로쪼개져 학습 문서가 된다. Apollo 계획에 관한 글에서 “Apollo 계획은케네디의 과학 자문 Jerome Wiesner 의 반대에 부딪혔다” 라는 문장이 나오고, 그 문장으로 LoRA SFT 를 돌린 모델이 원문 없이 “Jerome Wiesner” 를 답한다.

**형식 B — 갱신 규칙.** few-shot 실험에서 self-edit 은 데이터가아니라 **설정**이다. 모델이 `basic_augmentations: true, strategy: loss on all tokens, learning rate: 1e-05, epochs: 3` 같은 명세를 직접 쓴다. 회전 · 뒤집기 · 전치 같은 변환 도구의 선택도, 학습률과 epoch 수도, loss 를 전체 토큰에 걸지 출력 토큰에만 걸지도 생성대상이다.

**형식 B가 표준형에 대한 가장 날카로운 확장이다.** 식  $(U-\Theta)$  에서  $\text{gen}$  은 데이터 생성기로만 정의되어 있고학습률 · epoch 는 식 밖의 하이퍼파라미터였다. 형식 B 는 그하이퍼파라미터를  $\mathcal{R}_k$  안으로 끌어들인다.

루프는 라운드마다 다섯 단계다. (i) 모델이 self-edit 을 생성한다. (ii) 그것으로 SFT 를 돌려 임시 모델을 얻는다. (iii) 그 모델을 downstream 과제로 채점한다. (iv) **임시 모델을 버린다**. (v) 점수가 좋았던 self-edit 들만 남겨 그것을 쓴 정책을 민다. 배포 시에는 (i) – (ii) 만 돌리고결과를 남긴다.

**경고.** SEAL 이 말하는 “inner loop” 는 Neural Memory 계열이 말하는 inner loop 가 **아니다**. 후자는 토큰마다  $W$  를 미는 per-token 갱신이지만, SEAL 의 inner loop 는 **SFT 실행 한 번 전체**다. 걸리는시간이 초에서 분 단위다. 이 논문에 per-token 갱신은 어디에도 없다.

비유 학생이 시험 범위를 받고 스스로 요약 노트를 만든 다음, 그 노트로만 공부하고 시험을 본다. 점수가 좋으면 그 노트 만드는 방식을 강화한다. — 비유가 깨지는 곳: 학생은 노트를 만든 뒤에도 교과서를 다시 펼 수 있지만 여기서는 원문이 질의 시점에 사라진다. 갱신된 모델은 passage 를 보지 않고 답한다. 그리고 강화되는 것은 노트 자체가 아니라 노트를 쓰는 습관이다 — 임시 모델은 매번 버려지고 남는 것은 생성 정책뿐이다.

읽기 쪽은 이 권에서 가장 순수하다. wake 프롬프트에 검색 결과도 원문도 없이 “질문에 직접 간결하게 답하라. 질문: … 답:” 두 줄뿐이다. *W* 도 *E* 도 이 논문에 존재하지 않는다. 자세한 것은 ch20 § 20.3, § 20.3.2.

## 9. SEAL의 자기 표가 논지를 세 번깎는다

초록의 헤드라인은 “모델이 스스로 만든 데이터가 GPT-4.1 이 만든 합성데이터를 능가한다”이다. 표를 보면 그것이 참인 칸은 셋 중 하나다 — 단일 passage 에서 47.0 대 46.3, **마진 0.7 점**이다. 문서를 200 개 · 2067 개로 늘린 두 열에서는 GPT-4.1 이 이긴다 (59.4 대 58.2, 49.2 대 46.4). 본문은 이 사실을 진술하고 초록은 범위 조건을 달지 않는다.

열 사이를 스케일 계열로 읽어서도 안 된다. 단일 passage 는 LoRA 이고 나머지들은 full fine-tuning 이므로, 47.0 → 58.2 에는 문서 수의 변화와 갱신 방식의 변화가 함께 들어 있다. 뒤쪽 열의 하락도 방법의 열화로만 읽을 수 없다 — 평가 집합이 바뀌어 base 모델도 32.7 에서 29.0 으로 떨어진다. 읽을 수 있는 것은 base 대비 마진뿐이고, SEAL 은 25.5 에서 17.4 로, GPT-4.1 은 26.7 에서 20.2 로 줄어든다. **둘 다 줄되 SEAL 이 더 많이 준다.**

더 무거운 것은 부록의 프롬프트 민감도 표다.

표 E09-2. self-edit 프롬프트를 바꾸었을 때의 no-context SQuAD 정확도 [SEAL Table 10]

| 프롬프트                    | RL 없음       | 1 라운드 | 2 라운드       |
|-------------------------|-------------|-------|-------------|
| Implications (본문의 주 설정) | 39.7        | 43.7  | <b>47.0</b> |
| Implications-long       | <b>49.3</b> | 52.4  | 54.4        |
| Rewrite                 | <b>49.4</b> | 55.3  | 55.6        |
| Self-QA                 | 37.3        | 42.8  | 48.7        |
| No-Prompt (형식도 모델이 정함)  | 13.8        | 12.7  | 18.9        |

세 가지가 한 표에서 나온다. 첫째, **RL 을 한 번도 돌리지 않은 두 프롬프트 (49.3, 49.4) 가 논문의 RL 헤드라인 (47.0) 을 넘는다.** 프롬프트 한 단어를 바꾸는 것이 RL 루프 전체보다 값이 크다. 둘째, 그럼에도 **RL 의 기여는 실재한다** — 모든 행에서 라운드를 돌면 6 에서 11 점이 오르고, 좋은 프롬프트에서 출발하면 55.6 까지 간다. 문제는 기제가 아니라 본문이 고른 출발점이다. 셋째, “**스스로 적응한다**” 는 서사가 **마지막 행에서 반증된다.** 모델이 self-edit 의 형식까지 스스로 정하게 두면 RL 없이 13.8, 두 라운드 뒤에도 18.9 다. 자기 편집은 **저자가 고른 형식안에서만 작동한다.**

정의적 기여를 지워도 거의 잃지 않는다는 결과도 있다. 보상을 “실제로 fine-tune 하고 평가하기” 에서 “GPT-4.1 에 루브릭으로 채점시키기” 로 바꾸면 47.0 대 **45.6** 이고, 시간은 약 6 시간 대 **약 5 분**이다. 약 72 배의 절감에 1.4 점이다.

few-shot 결과에도 단서가 붙는다. ICL 0%, RL 없는 자기 편집 20%, SEAL 72.5%, 손으로 정한 설정 100%인데, **과제 19 개가 “손으로 정한설정 아래에서 풀릴 수 있도록” 필터되었다.** 따라서 100%는 실증 결과가 아니라 필터의 정의적 귀결이고, ICL의 0%도 ICL이 못 하는 과제만 고른 결과다. 그 위에서도 학습된 정책은 손으로 정한 설정에 27.5점 뒤진다.

망각은 등록만 되어 있다. 순차 self-edit 8회에 걸쳐 이전 과제 성능이 단조로 하락하는 히트맵이 있는데 **정확도 값 자체는 표로 인쇄되지 않고 표준오차만 인쇄되며**, 통제군도 없다. 그래서 이 권은 SEAL에 대해  $\rho$ 를 수치로 옮길 수 없다.

**시스템 모델링 관점** SEAL을 함대에 얹으면 무엇이 바뀌는가. decode에서는 아무것도 바뀌지 않는다 — 적응된 모델은 평범한 transformer이고 adapter 하나를 빼면 추가상태 읽기도 검색 hop도 없다. **그리고 그것이 이 설계의 전부이자 위험 전부다.**

**wake 프롬프트가 비어 있는 쪽으로 줄어든다.** 검색 시스템은 질의마다 prefill로 값을 치르지만 SEAL은 질의마다 0을 치르고 한 번에 전부를 치른다. 즉 **이 설계는 전부가 상각 베틀이다.** 그런데 질의당 절감의 크기는 원래 문맥에 있었을 passage 길이이고, 논문이 그 토큰 수를 적지 않는다. 그래서 이 권은 절감 수치를 인쇄하지 않는다.

**배치가 두 갈래로 갈린다.** 단일 passage 레짐은 문서마다 adapter를 따로 학습한다. 문서마다 가중치 상태가 생기므로 shared-weight batching이 깨지고 요청별 adapter 라우팅이 전제가 된다. continued pretraining 레짐은 반대다 — 모두가 한 벌의 가중치를 쓰므로 배치가 살아나지만 문서·사용자 단위 격리가 사라지고, 하필 그 레짐이 heuristic 기준선에 지는 자리다 (46.4 대 48.6). **Θ-경로 서빙 문제의 두뿔이며, 논문은 둘을 다 보고하면서 그 교환에 이름을 붙이지 않는다.**

**sleep job의 모양도 통상과 다르다.** 생성·평가에 vLLM, SFT에 DeepSpeed ZeRO-3를 쓴다. 이 함대는 문맥마다 학습 job을 하나씩 돌리며, 서빙 스택과 학습 스택이 같은 자리에 상주해야 한다.

**그런데 실제 걸림돌은 계산이 아니다.** RL 루프가 문맥마다 라벨된 downstream 과제를 요구한다. 서빙 함대에는 그런 라벨이 없고, 논문 자신이 이것을 라벨 없는 코퍼스로의 확장을 막는 요인으로 지목한다. 이 한 줄이 이 논문과 프로덕션 사이의 거리를 정한다.

자세한 것은 ch20 § 20.6, § 20.7.

## 10. 확장하고, 위로 증류하고, 아래를 지운다

Language Models Need Sleep은 이 권에서 유일하게  $W$ 에 쌓인 것을  $\Theta$ 로 옮기는 연산자를 명시적으로 가진 논문이다. 앞의 두 장이 각각 “어떻게 쓰는가”와 “무엇을 쓰는가”를 채웠다면, 이 논문은 “어디서 가져와 쓰는가”를 채운다.

wake 쪽 그림부터. 이 논문의 가중치는 한 덩어리가 아니라 **갱신주파수로 나뉜 블록의 사다리다.** 매 토큰 움직이는 끝 (KV cache나 고정 크기 memory)에서 아예 움직이지 않는 끝 (동결된 pre-trained MLP)까지 층층이 놓이고, 중간 블록들은 1,000·5,000·10,000 토큰마다 갱신된다. 이 갱신은 문맥 경계를 넘어 살아남지 않고 배포 아티팩트를 바꾸지도 않으므로  **$W$ 층 운동이다.** 주파수를 낮추는 것만으로는  $\Theta$ 에 닿지 못한다는 것이 앞 권의 판정이었고, 이 논문은 그 판정을 우회하지 않는다.

$\Theta$ 에 닿는 일은 별도의 오프라인 절차로 만든다. 한 번의 sleep은 세 조각이다.

**첫째, 받는 쪽 블록이 커진다.** 새 저랭크 expert 한 쌍  $(A, B)$ 가 추가된다.  $A$ 는  $d \times d_{\text{low}}$ ,  $B$ 는  $d_{\text{low}} \times d$  모양이고  $d_{\text{low}} \ll d$ 다. **증가중치의 개수 자체가 라운드마다 늘어난다.** 표준형

( $U-\Theta$ )는 고정 크기  $\Theta$ 에서 gradient를 빼는 식이므로, 이것이 표준형으로부터의 가장 큰 이탈이다. 왜 늘리는가에는 이유가 있다 — 이 논문은 catastrophic forgetting을 표집 분포의 문제가 아니라 **용량의 문제**로 재정의한다. 자리가 모자라서 덮어쓰게 되는 것이 원인이라면 답은 자리를 늘리는 것이다.

**둘째, 새로 생긴 자리에만 쓴다.**

**정의. Knowledge Seeding**은 확장 전의 자기자신을 teacher로, 확장 후의 자기 자신을 student로 삼아, 새로생긴 저랭크 expert에만 gradient를 흘려 teacher의 출력 분포와 표집 거동을 재현시키는 절차다. teacher가 student보다 **작다**는 점에서 고전적 distillation의 방향이 뒤집혀 있다.

나머지 파라미터는 전부 동결이다. 그래서 sleep이 끝나도 옛 가중치는 비트단위로 동일하다 — 별점으로 옛 지식을 지키는 정규화 계열보다 훨씬 강한비간섭 보장이며, 대가는 옛 가중치를 아예 건드리지 않는 것이다.

**셋째, 보내는 쪽을 지운다.** 보내는 블록에 그동안 쌓여 있던 저랭크 expert들을 리셋한다 (논문의 표현으로 synaptic pruning). **이 삭제가 있기 때문에 이 절차는 복사가 아니라 이송이다.** 그리고 이것이 상태량을 유계로 만드는 유일한 기제다.

순서도 기제의 일부다. 보내는 쪽의 갱신을 **계산만 하고 적용하지 않은 채** teacher/student를 세우고, 쓰기가 끝난 뒤에야 적용과 리셋을 함께 한다. 먼저 적용했다면 teacher는 이미 사라진 뒤라 증류할 표적이 없다. 즉 여기서 teacher는 다른 모델도 더 큰 모델도 아니라 **아직 덮어쓰이지 않은 몇 밀리초 전의 자기 자신**이다.

sleep의 두 번째 단계가 Dreaming이다.

**정의. Dreaming**은 모델이 과제 문맥을 조건으로 자기샘플을 생성하되 **일부러 무관한 expert를 하나 더 섞어 뽑고**, 생성된 후보를 gradient 크기로 거른 뒤 살아남은 것을 각각 격리된 모델사본에 LoRA SFT로 적용하는 절차다. 생성 정책은 SEAL과 같은 이진 개선보상으로 학습된다.

**비유** 하루치 메모를 저녁에 정서해 노트에 옮겨 적고, 옮긴 메모지는 버린다. 노트에는 매번 새 페이지를 끼운다. — **비유가 깨지는 곳:** 사람은 메모를 보고 옮겨 적지만 이 모델은 **원자료를 보지 않는다**. teacher에게서 새로 뽑은 표본으로 배우므로, 옮겨지는 것은 그날 실제로 있었던 일이 아니라 **직전의 자기 자신이 그 일에 대해 말하는 방식**이다. 실데이터 replay buffer가 이 설계에 없다.

자세한 것은 ch21 § 21.3, § 21.4.

## 11. 다리는 놓였다. 하중은 재지않았다

이 논문이  $\Theta$ -경로를 별도 축으로 세우는 근거를 실증한다는 것은 사실이다. 그리고 그 다리가 하중을 견디는지를 재는 값이  $\rho$ 인데, **그값은 없을 뿐 아니라 주어진 증거로부터 계산될 수도 없다**. 이유가 결정적이다 — **sleep 주기 수를 독립변수로 놓은 실험이 논문에 하나도 없다**. 통합 단계 수를 바꾼 실험은 있지만 그것은 사다리의 층수이지 라운드 수가 아니다. 반복되는 wake/sleep 수명주기를 주장하는 논문에서 반복이 측정되지 않았다.

나머지도 비어 있다.  $B_s$  는 절대값 없이 비율만 있고 (같은 step 수라면 SFT 가 4 배 효율적이고, 같은 성능 기준으로는 SFT 가 3.6 배에서 4.8 배의벽시계를 쓴다),  $L_w$  는 어떤 시간 수치도 없으며,  $C_{cap}$  은 바이트가 없다. 네 칸 중 절대값이 있는 칸이 0 개다. 그래서 이권은 이 논문에 대해 “효율적” 이라고 쓰지 않는다.

여기서 상각식도 퇴화한다. 쓰기가 문맥별이 아니라 스케줄별이고 수혜자가 이후의 모든 질의이므로, 분모가 “앞으로의 전 트래픽” 이 되어 상각된 sleep 비용이 0 으로 간다. 이것은 논문의 승리가 아니라 프레임에 대한 발견이다 — 상각식은  $E \cdot W$  -경로의 도구이고  $\Theta$  -경로는 그것을 받지 않는다. 그 순간 구속력 있는 질문이 돌로 옮겨간다. 쓰기가 살아남는가 ( $\rho$ ) 와 용량이 버티는가 ( $C_{cap}$ ). 이 논문은 그 둘을 다 보고하지 않는다.

기여의 배분도 표가 다르게 말한다. 논문 본문은 “모든 구성요소가 성능에 긍정적으로 기여한다” 고 쓰는데, 그 문장이 인용하는 표에서 **Semantic Reward** 를 빼면 AIME-25 가 69.0 에서 69.2 로 오른다. 하필 그 구성요소가 이름도 크기도 밝혀지지 않은 외부 frozen reward model 에 의존하는 항이고, 효율 비교에도 그 모델의 비용이 계상되어 있지 않다. 더 날카로운 것은 확장의 뭉치다. 최강 기준선에 구조적 확장만 없으면 76.6/67.4/45.1 이 77.9/68.2/45.9 로 오르는데, 전체 마진이 +2.6/+1.6/+1.0 이다. **마진의 절반가량 — 한 벤치마크에서는 1.0 중 0.8 — 이 Knowledge Seeding 없이 파라미터를 더준 것만으로 얻어진다.** 지식 주입 쪽에서는 반대로 Dreaming 이 결과를 지고 있다 (제거하면 - 13.2/- 10.0). 즉 이 논문의 이름이 걸린 기여가 자기 뭉치를 수치로 보인 자리가 없다.

표의 신뢰구간도 없다. 시드도 오차막대도 논문 전체에 없는데 논의되는 차이의 다수가 0.2-1.5 점이고, 일곱 실험 중 넷은 아예 숫자 없이 그림으로만 보고된다. few-shot 헤드라인 80%는 32/40 이고 비교 대상 72.5%는 29/40 이므로 격차가 결과 3 개다. 그리고 이 논문의 두 기여가 자기 아키텍처 위에서 함께 돈 실험이 없다 — 통합은 사다리 위에서, Dreaming 은 평범한 transformer 위에서 검증되었고 합성은 미검증이다.

**시스템 모델링 관점** 이 설계가 서빙 엔진에 요구하는 것은 앞의 두 장과 종류가 다르다. 첫째, **decode 경로 안에 backward 가 상주해야 한다.** 사다리의 유한 주파수 블록은 토큰마다 optimizer 오차를 누적하고 1,000 토큰마다 가중치에 적용한다. 활성 블록마다 파라미터 크기의 accumulator 가 하나씩 필요하고, 1K 토큰마다 가중치 쓰기가 발생한다. **inference 가 forward 전용이 아니게 되며, 그 변화가 sleep 이 아니라 wake 에 착지한다.** 논문은 이 wake 비용에 어떤 숫자도 붙이지 않는다.

둘째, **텐서 모양은 일부러 고정된다.** 파라미터를 실제로 늘리면 컴파일된 그래프와 kernel autotuning 과 checkpoint 레이아웃이 매 sleep 마다 깨지므로, 구현은 **처음부터 다 만들어 두고 마스킹한다.** 활성화는 sleep 때 마스크를 푸는 것으로 이루어진다. 이 corpus 에서 가장 서빙 친화적인 설계 결정이며, 대가는  $t = 0$  에 정해지는 하드 천장이다 — **그 pool 크기가 논문에 없고, 천장에 닿으면 무슨 일이 일어나는지도 없다.**

셋째, **이 상태는 전역이다.** 논문은 모델 하나를 유지하고 사용자별·세션별 가중치를 만들지 않는다. 따라서 “ $\Theta$  -경로는 per-user delta 를 만들고 그 delta 트래픽이 새 기회를 연다” 는 서사를 이 논문에 귀속시킬 수 없다. 이 논문이 실제로 공급하는 문제는 per-user 상태가 아니라 **버전 관리다** — sleep 이 주기마다 발화하고 매 발화가 갱신·활성화·리셋을 수행하므로 checkpoint 관리·캐시 무효화·회귀테스트·롤백이 전부 주기 단위 운영이 된다. 게다가 **리셋의 목표값이 명시되어 있지 않아**, 외부 스냅샷 없이는 순진한 구현이 sleep 이전 모델로 되돌아갈 방법이 없다.

넷째, **few-shot 프로토콜은 배치를 요청 안에서 깨뜨린다.** 과제 하나에 dream 5 개를 만들고 각각을 독립 적용한 뒤 예측하므로, LoRA 로 갈라진 모델 인스턴스 5 개와 그 안의

fine-tuning 5 회가 **사용자 응답경로 안에** 들어간다. 붙은 지연 수치는 없다. 그리고 이 sleep job 의 지배적비용 모양은 training FLOPs 가 아니라 **autoregressive decode** 다 (과제당 60 회 생성 + 60 회 backward + 15 회 LoRA 적합). 그러면 sleep job 은 훈련 클러스터가 아니라 wake 트래픽과 **같은 하드웨어, 같은 병목**을 놓고 경쟁한다.

자세한 것은 ch21 § 21.5, § 21.6, § 21.7.

## 12. 침묵의 지도

이 권의 논문들은 서로를 이상할 만큼 모른다.

**같은 층 안에서도 모른다.** LoRA 는 ROME 보다 8 개월 앞서 나왔고 둘다 같은 종류의 저랭크 가중치 쓰기인데, ROME 은 LoRA 를 인용하지 않고 자기갱신을 1972 년의 associative memory 문헌에 연결한다. MEMIT 도 PEFT 계보를한 번도 언급하지 않는다. **두 논문은 서로를 모른 채 같은 공간을 반대 방향에서 채운다** — LoRA 는 rank 를 설계 변수로 고정하고, MEMIT 의 rank 는 편집 개수의 부산물로 결정된다. 지속학습 공동체의 어휘도 쓰이지 않는다. MEMIT 이 옛 지식을 2 차 통계로 보호하는 항은 기능적으로 EWC 의 Fisher 정규화와 같은 자리에 있지만 EWC 를 인용하지 않는다.

**SEAL 은 다른 경로의 존재를 모르는 가장 강한 사례다.** 논문의 전망절은 “확장된 상호작용에 걸쳐 작동하며 상호작용 뒤 가중치 갱신을 촉발하는 self-edit 을 합성하는 에이전트”를 목표로 제시한다. **그것은 E-경로의 문제 진술 그 자체이고, E-경로는 그 문제를 가중치가 아니라 레코드에 써서 푼다.** SEAL 은 그 대안을 인용하지도 비교하지도 않고, 왜 레코드 대신 가중치여야 하는지 논증하지 않는다. 연대도 변명이 되지않는다 — 상대는 동시대이거나 1 년 반 앞서 나왔다. 그리고 이 논문은 sleep-time 의 문제에 전부 도달하면서 “**sleep**” 이라는 단어를 한 번도 쓰지 않는다.

**Language Models Need Sleep 의 침묵은 더 정밀하다.** W-경로의 아키텍처 계보를 전수 인용하고 그중 일부를 실제 기준선으로 돌리기까지 하는데, **제목이 한 글자 차이로 겹치는 W-경로의 sleep 논문만 참고문헌에 없다.** 그쪽은 이 논문을 인용하고 있으므로 침묵은 일방향이다. E-경로는 이름으로 부르되 두 문장으로 닫는데, **기각이 정의로 이루어지고 실험으로 이루어지지 않았다** — E-경로 시스템은 일곱 실험 어디에도 기준선으로 등장하지 않는다.

**비유** 같은 부품을 세 공장이 서로 모른 채 따로 만드는 상황이다. — **비유가 깨지는 곳:** 공장은 특허와 원가로 자기가 중복했다는 사실을 결국 알게 되지만 여기서는 아무도 모른 채 끝난다. 유일한 실제 연결점은 사후적이다 — MEMIT 이 선언한 “선형 층을 associative memory 로 본다” 는관점의 두 출처 (Kohonen 1972, Anderson 1972) 가 W-경로 계보의 공통조상이기도 하다. **두 경로는 서로를 인용하지 않으면서 같은 1972 년조상을 공유한다.**

자세한 것은 ch19 § 19.8.3, ch20 § 20.8, ch21 § 21.8.3.

## 요약

- $\Theta$ -경로는 세 층 중 유일하게 **배포 아티팩트를 바꾼다**. 이 한문장에서 개인화 가능성·배칭 가능성·롤백단위·캐시 수명이 전부 걸린다.
- 두 쓰기 방식을 가르는 것은 rank 도 정확도도 아니라 **아티팩트를 남기느냐**. 둘 다 wake 지연을 0으로 만들지만, delta 는 되돌리기·개인화·다중 배포를 얻는 대신요청마다 다른 가중치를 감수하고, 직접 편집은 shared-weight batching 을 지키는 대신 되돌리기와 개인화를 통째로 포기한다. **개인화하려면 delta 여야 한다**.
- ROME 의 갱신에는 **학습률이 없다**. 단 한 해 한 번이며,  $\Theta$ -경로의 쓰기가 반드시 SGD 여야 하는 것은 아니라는 존재증명이다. 그리고 쓸자리는 학습되지 않고 계층 결과를 사람이 읽어 고정한다.
- 편집 용량의 실증치는 GPT-J 6B 에 10,000 개 삼중항이고, 그 지점에서 **견드리지 않은 이웃의 정답률이 83.5 에서 73.7 로 떨어진다**. 10,000 은 붕괴점이 아니라 관측된 하한이다. zsRE 에서는 이웃 보존 26.6 이 편집하지 않은 모델의 27.0 보다 낮다.
- 편집당 2.6773 초는 **쓰기 연산의 값이지 올바르게 썼는지 확인하는 값이 아니다**.  $\Theta$ -경로에서 쓰기가 싸다는 인상은 검증을 회계 밖에 둔 결과다. 그리고 ch19 의 세 도구는 **무엇을 쓸지를 아무것도 제공하지 않는다** — 학습 집합이 언제나 밖에서 오고, sleep 라운드가  $k = 1$  에서 멈추므로  $\rho$  를 켈 실험도 없다.
- SEAL 은 그 빈칸에 모델 자신을 놓는다. self-edit 의 실증된 형식은 둘 — 합의 목록(데이터) 과 augmentation·학습률·epoch 명세(갱신규칙)이며, 후자가 표준형에 대한 가장 날카로운 확장이다.
- **SEAL 의 자기 표가 논지를 세 번 깎는다**. RL 없는 프롬프트들 (49.3, 49.4) 이 RL 헤드라인 (47.0) 을 넘고, 정의적 기여인 학습 기반보상을 채점 호출로 바꿔도 45.6 에 약 72 배 싸며, 모델이 형식까지 정하면 18.9 로 무너진다. 초록의 “GPT-4.1 능가” 는 세 열 중 한 열에서만 성립한다.
- Language Models Need Sleep 은 corpus 에서  $W \rightarrow \Theta$  **이송 연산자를 명시적으로 가진 유일한 논문**이다. 확장하고, 확장 전 자기 자신에게서 위로 증류하고, 보내는 쪽을 지운다. 마지막의 삭제가 이것을 복사가 아니라 이송으로 만든다.
- 그러나 **하중은 재지 않았다**. sleep 주기 수를 독립변수로 놓은 실험이 하나도 없어  $\rho$  가 계산조차 되지 않고, 비용 네 칸에 절대값이 0 개다. 마진의 절반가량은 Knowledge Seeding 이 아니라 파라미터를 더 준 것에서 나오며, 한 구성요소는 제거했을 때 성능이 오른다 (69.0  $\rightarrow$  69.2).
- $\Theta$ -경로에서는 상각식이 되화한다. 쓰기가 문맥별이 아니라 스케줄별이라 분모가 “앞으로의 전 트래픽” 이 되기 때문이다. **구속력 있는 질문이 상각에서  $\rho$  와  $C_{\text{cap}}$  으로 옮겨가고, 이 권의 논문 중 그 둘을 보고한 것이 없다**.

## 자가 점검

- delta 방식과 직접 편집을 산출물의 형태로 정의하고, 두 방식이 같은  $L_w = 0$  을 서로 다른 대가로 산다는 것을 배포 단위와 배칭 두 향으로 설명할 수 있다.
- “직접 편집으로는 사용자별 개인화가 구조적으로 불가능하다” 는 문장을 한문단으로 풀어 쓸 수 있다.
- ROME 의 쓰기에  $\eta_\Theta$  가 없다는 것이 무슨 뜻인지 말하고, 그럼에도 남아 있는 학습률 (0.5) 이 무엇을 미는 학습률인지 구별할 수 있다. 그리고 10,000 이 왜 상한이 아니라 하한인지, 그 값이 어느 지표를 얼마나 희생한 값인지 수치로 딸 수 있다.

- self-edit의 두 형식을 구분하고, 형식 B가 식  $(U-\Theta)$ 의 어느자리를 넓히는지 말할 수 있다. 그리고 SEAL의 “inner loop”가 Neural Memory 계열의 inner loop와 어떻게 다른지 한 문장으로 말할 수 있다.
- 표 E09-2에서 RL의 기여와 프롬프트의 기여를 분리해 진술하고, 마지막행이 “스스로 적응한다”에 대해 무엇을 반증하는지 말할 수 있다.
- Language Models Need Sleep의 sleep 세 조각을 순서대로재구성하고, 왜 보내는 쪽 갱신의 적용을 미뤄야 하는지, 그리고 왜마지막의 삭제가 이 절차를 이송으로 만드는지 설명할 수 있다.
- 이 권의 논문들에 상각식을 세울 수 없는 이유를 분모의 정의로 설명하고, 그 결과 구속력 있는 질문이 어느 두 값으로 옮겨가는지 말할 수 있다.

## 다음 권으로

이 권은 세 경로를 전부 돌았다. 그리고 세 권이 같은 자리에서 멈춘다 —  $E$ -경로는 저장소가 무한히 자라도 되는지 재지 않았고,  $W$ -경로는 고정 크기 상태에 무엇이 남는지 재지 않았으며,  $\Theta$ -경로는 **반복해서써도 되는지**를 재지 않았다. 세 번 다 같은 칸,  $\rho$ 와  $C_{cap}$ 이다.

다음 권부터는 논문을 한 편씩 읽는 대신 그 빈칸을 정면으로 묻는다. 가중치에 사실을 계속 넣을 수 있는가, 자기가 만든 데이터로 자기를 계속학습시키면 무엇이 무너지는가, 그리고 세 경로의 비용을 같은 자로 나란히놓으면 무엇이 남는가. 이 권이 넘기는 것은 도구가 아니라 **두 개의빈칸**이다.